Final Project Report: Superpixels

Vinícius da Mata e Mota Felipe Arruda Brito
CentraleSupélec

Abstract

Superpixels provide a compact and structure-preserving image representation that reduces computational complexity while maintaining perceptually meaningful boundaries. In this work, we present a complete implementation and analysis of the Simple Linear Iterative Clustering (SLIC) algorithm from first principles, together with a systematic comparison against widely used baselines, including OpenCV SLIC, SEEDS, and a graph-based segmentation method.

Our implementation details feature representation in CIELab space, localized color–spatial distance computation, iterative clustering with bounded search regions, and a modified connectivity enforcement strategy that merges stray components into the largest adjacent region. We evaluate methods on the BSDS500 benchmark using Boundary Recall (BR), Undersegmentation Error (UE), Achievable Segmentation Accuracy (ASA), and runtime measurements.

Experimental results show improvements in segmentation quality as the number of superpixels increases, with our custom SLIC achieving competitive or superior boundary adherence compared to OpenCV implementations at medium and high K. While optimized C++ backends maintain a significant runtime advantage, our implementation demonstrates that high-quality superpixels can be obtained with a clear and reproducible algorithmic design. The study highlights practical trade-offs between segmentation accuracy and computational efficiency, and provides insights for future improvements in superpixel generation.

1. Introduction and Motivation

Modern computer vision systems operate on increasingly high-resolution images, often containing millions of pixels. While this fine-grained representation preserves visual detail, it also introduces substantial computational cost and redundancy. Many vision tasks—such as semantic segmentation, object recognition, tracking, and image editing—do not require reasoning at the individual pixel level. Instead, they benefit from intermediate representations that capture perceptually meaningful structures while reducing data complexity. Superpixels address this need by grouping neighboring pixels with similar appearance into coherent atomic regions, providing a compact and structure-preserving image representation.

An effective superpixel algorithm must satisfy several desirable properties: boundary adherence to respect object contours, region homogeneity to ensure perceptual consistency, computational efficiency for large-scale applications, and robustness to noise, texture variation, and illumination changes. Over the past two decades, numerous superpixel generation methods have been proposed, including graph-based segmentation approaches [4], clustering-based techniques, and energy-optimization frameworks. Among these, the Simple Linear Iterative Clustering (SLIC) algorithm [1] has emerged as a widely adopted standard due to its simplicity, efficiency, and competitive performance. By formulating superpixel extraction as a localized clustering process in a joint color–spatial space, SLIC provides a favorable trade-off between computational cost and segmentation quality. Alternative methods such as SEEDS [2] adopt hierarchical energy optimization strategies that often improve boundary adherence but at the expense of increased algorithmic complexity.

Despite their widespread adoption, existing superpixel methods exhibit several limitations that motivate further investigation. First, performance is often sensitive to parameter choices, particularly the number of superpixels and compactness constraints. Second, many algorithms struggle with weak or blurry object boundaries, leading to leakage across semantic regions. Third, highly textured or heterogeneous areas may produce irregular or fragmented segments, reducing their usefulness for downstream tasks. These challenges highlight the need for a deeper understanding of the theoretical and practical behavior of superpixel algorithms and their impact on segmentation quality and computational performance.

This work aims to provide a systematic study of superpixel generation through the implementation and analysis of SLIC from first principles. By reconstructing the algorithm and comparing it with established implementations and alternative methods, we seek to characterize its strengths, limitations, and practical trade-offs. Through quantitative evaluation on benchmark datasets and analysis of key parame-

ters, the project investigates how design choices influence boundary preservation, region consistency, and runtime efficiency. Ultimately, this study contributes to a clearer understanding of superpixel representations and explores directions for improving their robustness and applicability in modern vision pipelines.

2. Problem Definition

Let $I : \Omega \to \mathbb{R}^3$ denote a color image defined over a discrete pixel lattice $\Omega \subset \mathbb{Z}^2$, where each pixel $p \in \Omega$ is associated with a color vector $I(p)$ represented in the CIELab color space. The objective of superpixel segmentation is to partition the image domain into a set of K non-overlapping, connected regions $\mathcal{S} = \{S_1, S_2, \ldots, S_K\}$ such that:

$$\bigcup_{k=1}^{K} S_k = \Omega, \qquad S_i \cap S_j = \varnothing \text{ for } i \neq j. \tag{1}$$

Each region S_k is referred to as a superpixel and represents a group of spatially contiguous pixels with similar visual characteristics.

2.1. Desired Properties

An effective superpixel decomposition should satisfy the following criteria:

- **Appearance Consistency:** Pixels within the same superpixel should exhibit similar visual properties such as color or intensity.
- **Spatial Coherence:** Superpixels should form compact and spatially localized regions.
- **Boundary Preservation:** Superpixel boundaries should align closely with perceptually meaningful image contours.
- **Computational Efficiency:** The segmentation process should scale favorably with image resolution.

2.2. Constraints

The partition is subject to the following constraints:

- **Connectivity:** Each region S_k must form a connected component in the image grid.
- **Cardinality:** The number of superpixels K should preferably be specified by the user.
- **Coverage:** Every pixel must belong to exactly one superpixel.

2.3. Evaluation Perspective

Given a segmentation $\mathcal{S}$, the quality of the partition is assessed by how well it preserves perceptually meaningful structures while reducing representation complexity. Performance is therefore evaluated in terms of boundary adherence, region consistency, and computational efficiency.

3. Related Work

Superpixel segmentation has become a fundamental preprocessing step in many computer vision pipelines, providing a compact and structure-preserving representation of images. Existing methods can be broadly categorized into graph-based approaches, clustering-based techniques, and energy-driven optimization methods.

3.1. Graph-Based Segmentation

Early work on superpixel generation is closely related to general image segmentation methods. The graph-based algorithm of Felzenszwalb and Huttenlocher [4] models the image as a weighted graph where pixels correspond to nodes and edges encode local similarity. Regions are formed through an efficient merging process guided by internal difference and boundary evidence. This method is computationally efficient and capable of preserving strong boundaries; however, it does not explicitly control the number, size, or regularity of segments, which limits its suitability for applications requiring uniform superpixel structures.

3.2. Clustering-Based Superpixels

Clustering-based approaches explicitly aim to produce compact and approximately uniform regions. Among these methods, Simple Linear Iterative Clustering (SLIC) [1] has become one of the most widely adopted algorithms. SLIC formulates superpixel extraction as a localized clustering process in a joint color–spatial space, resulting in an algorithm with linear computational complexity and a small number of parameters. Its simplicity, efficiency, and strong empirical performance have made it a standard baseline in many studies and practical implementations.

Despite these advantages, SLIC exhibits several limitations. Its performance depends on the choice of compactness and superpixel number parameters, and it may struggle to preserve weak or low-contrast boundaries. Additionally, highly textured regions can lead to irregular segment shapes or boundary leakage.

3.3. Energy-Driven Superpixel Methods

Energy-driven approaches formulate superpixel extraction as an optimization problem over region assignments. SEEDS [9] adopts a hierarchical block-based updating strategy guided by an energy function that promotes color homogeneity within regions. This method achieves strong boundary adherence and real-time performance through efficient histogram-based updates. However, the hierarchical optimization procedure increases algorithmic complexity and introduces additional design choices compared to clustering-based approaches.

3.4. Evaluation of Superpixel Algorithms

The evaluation of superpixel methods has been systematically studied through benchmark frameworks such as the one proposed by Neubert and Protzel [8]. Standard metrics include Boundary Recall, which measures alignment with ground-truth contours; Undersegmentation Error, which quantifies leakage across object boundaries; and Achievable Segmentation Accuracy, which estimates the best possible labeling accuracy given a superpixel partition. These metrics capture complementary aspects of segmentation quality and enable fair comparison across methods.

3.5. Position of This Work

This work focuses on the implementation and analysis of SLIC from first principles, together with a comparative evaluation against established implementations and alternative segmentation approaches. By examining both segmentation quality and computational efficiency under controlled experimental conditions, the study aims to characterize the practical trade-offs of widely used superpixel methods and to better understand their behavior across different image characteristics.

Motivated by the strong empirical performance and widespread adoption of SLIC, this work provides a detailed implementation and analysis of the algorithm from first principles. The methodology section describes the feature representation, distance formulation, iterative optimization procedure, and connectivity enforcement used to generate superpixels. This formulation enables a systematic evaluation of segmentation quality and computational efficiency under controlled parameter settings.

4. Methodology

This section presents the formulation and implementation of the Simple Linear Iterative Clustering (SLIC) algorithm for superpixel generation. The method produces compact and approximately uniform regions by performing localized clustering in a joint color–spatial feature space.

4.1. Feature Representation

Let $I : \Omega \to \mathbb{R}^3$ denote an image defined on a discrete pixel grid $\Omega \subset \mathbb{Z}^2$. Each pixel $p = (x_p, y_p) \in \Omega$ is represented by a five-dimensional feature vector:

$$\mathbf{f}(p) = [L_p, a_p, b_p, x_p, y_p]^T, \tag{2}$$

where (L_p, a_p, b_p) are color components in the CIELab space and (x_p, y_p) denote spatial coordinates.

Given a desired number of superpixels K, the approximate spacing between cluster centers is defined as:

$$S = \sqrt{\frac{|\Omega|}{K}}. \tag{3}$$

4.2. Distance Measure

Each superpixel is associated with a cluster center $\mathbf{c}_k = [L_k, a_k, b_k, x_k, y_k]^T$. Pixel assignment is determined by a distance measure that balances color similarity and spatial proximity:

$$d_{lab}^2 = \|\mathbf{f}(p)^{\text{color}} - \mathbf{c}_k^{\text{color}}\|_2^2 \tag{4}$$

$$d_{xy}^2 = \|\mathbf{f}(p)^{\text{spatial}} - \mathbf{c}_k^{\text{spatial}}\|_2^2 \tag{5}$$

$$D(p, \mathbf{c}_k) = \sqrt{d_{lab}^2 + \left(\frac{m}{S}\right)^2 d_{xy}^2} \tag{6}$$

where m is the compactness parameter controlling the trade-off between boundary adherence and spatial regularity.

4.3. Cluster Initialization

Cluster centers are initialized on a regular grid with spacing S. To avoid placing centers on strong edges, each initial position is perturbed to the lowest gradient location within a local neighborhood.

4.4. Localized Clustering

Unlike standard k-means, SLIC restricts the search region of each cluster center to a square window of size $2S \times 2S$. This search area size is inspired heuristically by the fact that the expect superpixel is of size $S \times S$. This constraint is crucial since it reduces computational cost, allowing for a linear complexity and reduced memory requirement, while preserving segmentation quality.

The algorithm iteratively alternates between:

- Assigning pixels to the nearest cluster center according to the distance measure,
- Updating cluster centers as the mean feature vector of assigned pixels.

Iterations continue until convergence or until a fixed number of iterations is reached. From empirical experimentation, the literature on the topic asserts that 10 iterations is generally sufficient for most images [1].

4.5. Connectivity Enforcement

Although the spatial term in the distance measure promotes spatial coherence, it does not guarantee that each superpixel forms a connected component. As a result, small isolated pixel groups may appear after clustering. To ensure spatial consistency, a post-processing step enforces connectivity by relabeling such stray components to one of their neighboring superpixels, thereby eliminating fragmented regions.

In the original SLIC formulation [1], each disconnected component is reassigned to the neighboring superpixel with the smallest distance in the feature space. In contrast, we propose an alternative assignment strategy in which stray

components are merged with the largest adjacent superpixel. This modification favors region stability and reduces the creation of small irregular segments.

Both implementations have a complexity of $O(N)$, where $N = |\Omega|$, with only a additional $O(K)$ extra cost in memory for our proposed approach, which keeps the algorithm's overall complexity linear.

4.6. Computational Complexity

A key advantage of SLIC is its computational efficiency, which arises from restricting the clustering procedure to local spatial neighborhoods. In contrast to standard k-means, where each pixel may be compared to all cluster centers, SLIC limits the assignment step to a fixed search region around each center. Given an expected superpixel spacing $S = \sqrt{N/K}$, each cluster center only considers pixels within a $2S \times 2S$ window. As a consequence, each pixel is compared to only a small, constant number of nearby cluster centers (empirically fewer than eight), thereby avoiding a large number of redundant distance computations.

Because the size of the local search region does not grow with either image resolution or the number of superpixels, the number of distance evaluations per pixel remains bounded. The overall computational cost of each iteration is therefore proportional to N, and the total complexity of SLIC is linear in the number of pixels. Importantly, this complexity is effectively independent of the number of superpixels K, as increasing K reduces the search region proportionally.

By comparison, classical k-means clustering has a trivial upper bound of $O(K^N)$ per iteration [7], and a practical runtime of $O(NKI)$, where I denotes the number of iterations required for convergence [3]. Several general acceleration strategies for k-means have been proposed, including random sampling [6], local search heuristics [5], and bounding techniques exploiting the triangle inequality [3]. While effective in generic clustering settings, these approaches are not tailored to the spatial structure of images.

In contrast, SLIC is specifically designed for superpixel generation, exploiting spatial locality to achieve linear-time performance while maintaining high segmentation quality. This property distinguishes SLIC from many alternative superpixel algorithms whose computational cost depends explicitly on the number or size of regions.

5. Evaluation and Results

5.1. Experimental Protocol

We evaluated the methods on the BSDS500 benchmark test split, using the official human segmentations as ground truth (`.mat` files). Each image contains multiple annotations, and we computed the metrics against every annotation and then averaged the values per image. The plots shown in this report use representative test images (`100007.mat` and `100039.mat`), with trends consistent with the full evaluated subset.

Algorithm 1 Efficient Superpixel Segmentation (SLIC)

Initialize cluster centers $\mathbf{c}_k = [L_k, a_k, b_k, x_k, y_k]^T$ by sampling pixels at regular grid steps S
Perturb cluster centers in an 3×3 neighborhood to the lowest gradient position
$i \leftarrow 0$
repeat
  for each cluster center $\mathbf{c}_k$ **do**
    Assign closest pixels in $2S \times 2S$ neighborhood
  end for
  Compute new cluster centers
  $i \leftarrow i + 1$
until $i \geq M$
Enforce connectivity
return superpixel partition

We compared four segmentation methods: Custom SLIC (our implementation) with compactness $m \in \{10, 20\}$, OpenCV SLIC, OpenCV SEEDS, and GraphSeg (OpenCV graph segmentation). For methods controlled by superpixel count, we used $K \in \{50, 100, 200\}$. GraphSeg is included as a fixed-parameter reference baseline.

We report the wall-clock execution time (Runtime) alongside Boundary Recall (BR), Undersegmentation Error (UE), and Achievable Segmentation Accuracy (ASA), as previously defined in Section 3.4.

5.2. Implementation Highlights with Code Snippets

The localized assignment strategy (Section 4.4) is optimized using NumPy slicing to evaluate centers strictly within their $2S \times 2S$ windows, maintaining practical linear complexity.

Listing 1. Localized assignment in custom SLIC.

```
for idx, (cL, ca, cb, ci, cj) in enumerate(centers):
    min_y = max(0, ci - S)
    max_y = min(h, ci + S + 1)
    min_x = max(0, cj - S)
    max_x = min(w, cj + S + 1)

    L_slice = L[min_y:max_y, min_x:max_x]
    a_slice = a[min_y:max_y, min_x:max_x]
    b_slice = b[min_y:max_y, min_x:max_x]
    y_slice = y_coords[min_y:max_y, min_x:max_x]
    x_slice = x_coords[min_y:max_y, min_x:max_x]

    dc_sq = (L_slice - cL)**2 + (a_slice - ca)**2 +
     (b_slice - cb)**2
    ds_sq = (y_slice - ci)**2 + (x_slice - cj)**2
    D = np.sqrt(dc_sq + ((m / S)**2) * ds_sq)

    current_dists = distances[min_y:max_y,
     min_x:max_x]
    mask = D < current_dists
    distances[min_y:max_y, min_x:max_x][mask] =
     D[mask]
```

```
    labels_slice = labels[min_y:max_y, min_x:max_x]
    labels_slice[mask] = idx
    labels[min_y:max_y, min_x:max_x] = labels_slice
```

The connectivity post-processing (Section 4.5), which merges stray components into the largest adjacent label, is implemented using OpenCV's connected components as follows:

Listing 2. Connectivity post-processing.

```
label_list = np.unique(labels)

for label in label_list:
    mask = (labels == label)
    num_disjoint, disjoint_labels =
     cv.connectedComponents(
        mask.astype(np.uint8)
    )

    if num_disjoint > 2:
        disjoint_sizes = {}
        for j in range(1, num_disjoint):
            disjoint_sizes[j] =
     np.sum(disjoint_labels == j)

        max_size = max(disjoint_sizes.values())
        for j in range(1, num_disjoint):
            if disjoint_sizes[j] < max_size:
                labels[disjoint_labels == j] = -1

stray_pixels = (labels == -1)
label_sizes = {lab: np.sum(labels == lab) for lab in
     label_list}
num_stray, stray_labels =
     cv.connectedComponents(stray_pixels.astype(
np.uint8))

for i in range(1, num_stray):
    mask = (stray_labels == i)
    max_label = largest_neighbour(mask, labels,
     label_list, label_sizes)
    labels[mask] = max_label
```

Finally, the evaluation pipeline computes the metrics per ground-truth segmentation and stores mean values for each method/configuration.

Listing 3. Boundary Recall, UE and ASA.

```
def boundary_recall(ground_truth_mask, boundary_mask,
     d=2):
    if np.sum(ground_truth_mask) == 0:
        return 0.0

    gt = (ground_truth_mask > 0).astype(np.uint8)
    bd = (boundary_mask > 0).astype(np.uint8)
    dist_map = cv.distanceTransform(1 - bd,
     cv.DIST_L2, 3)

    gt_pixels = (gt == 1)
    hits = np.sum(dist_map[gt_pixels] <= d)
    return hits / np.sum(gt_pixels)


def undersegmentation_error(ground_truth_labels,
     superpixel_labels):
    N = ground_truth_labels.size
    total_error = 0

    for s_label in np.unique(superpixel_labels):
        s_mask = (superpixel_labels == s_label)
        s_size = np.sum(s_mask)
        gt_in_sp = ground_truth_labels[s_mask]
        k_j = len(np.unique(gt_in_sp))
        total_error += s_size * k_j

    return (total_error - N) / N


def achievable_segmentation_accuracy(
ground_truth_labels, superpixel_labels):
    N = superpixel_labels.size
    sum_max_overlaps = 0

    for s_label in np.unique(superpixel_labels):
        s_mask = (superpixel_labels == s_label)
        gt_in_sp = ground_truth_labels[s_mask]
        if gt_in_sp.size == 0:
            continue
        _, counts = np.unique(gt_in_sp,
     return_counts=True)
        sum_max_overlaps += np.max(counts)

    return sum_max_overlaps / N
```

5.3. Quantitative Results on Representative Test Images

5.4. Discussion of the Curves

Increasing K consistently improves segmentation quality for all K-controlled methods: ASA increases, BR increases, and UE decreases. This behavior follows expected superpixel dynamics: finer partitions capture boundaries better and reduce region leakage. The cost is increased runtime, especially for the custom implementation.

For both representative images, `Custom_m10` and `Custom_m20` remain strong in ASA and BR as K increases. At $K = 200$, ASA reaches approximately 0.97 (`100007`) and 0.95 (`100039`) for the best custom setting. BR also increases steadily (about 0.70 $\rightarrow$ 0.87 on `100007` for `Custom_m10`).

The two compactness settings show a clear trade-off: `m=10` tends to favor boundary adherence (higher BR), while `m=20` usually provides slightly more regular regions and occasionally comparable or higher ASA at intermediate K. Both settings are valid; m should be selected based on downstream priorities.

OpenCV SLIC and SEEDS are extremely fast in this setup. Quality behavior is image-dependent: SEEDS can approach custom ASA/UE at high K, while OpenCV SLIC remains competitive in UE for some cases (e.g., `100039` at $K = 200$). The custom method gives stronger BR in these examples, suggesting effective contour localization in the implemented distance + connectivity combination.

GraphSeg provides strong boundary-related scores in these examples and very fast runtime. Its curve is horizontal in K-plots because the algorithm is configured with fixed graph parameters ($\sigma, k, \text{min\_size}$), so it serves as a reference rather than a directly matched K-controlled superpixel generator.

The custom implementation is one to two orders of magnitude slower than optimized C++ backends. The main bottlenecks are: Python-level loops in assignment/updates, repeated masking operations, per-label connectivity checks,

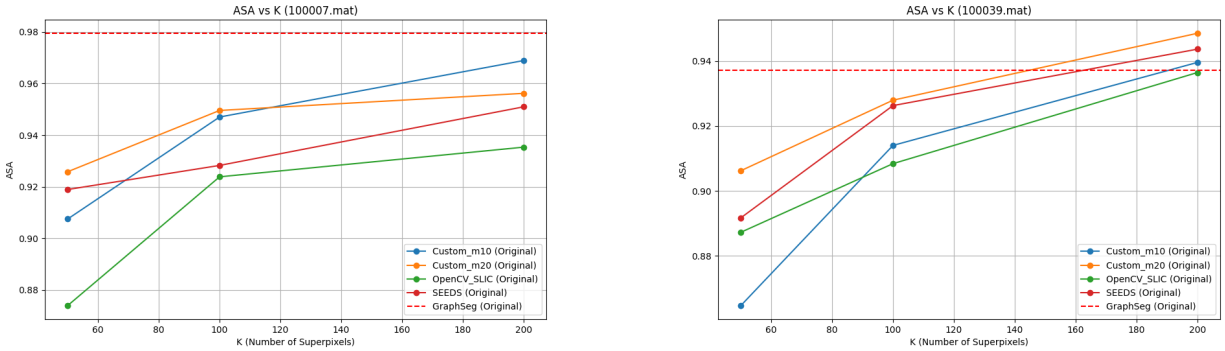

Figure 1. ASA vs. K for two representative BSDS500 test images.

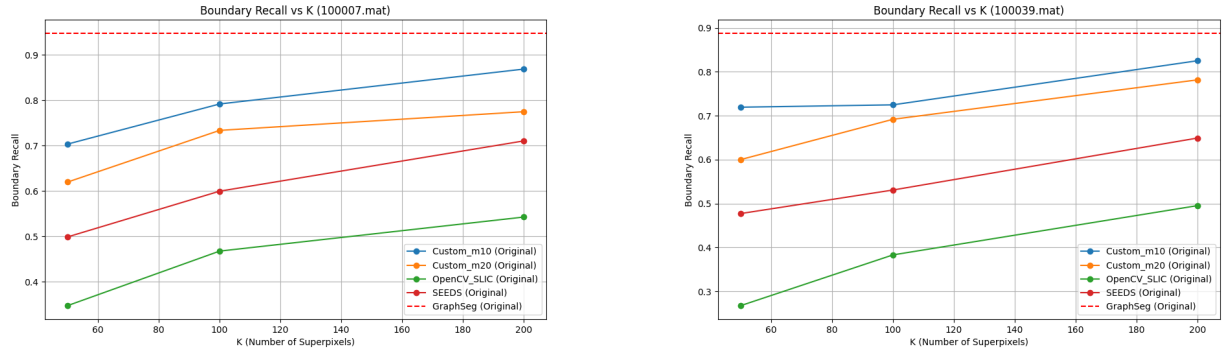

Figure 2. Boundary Recall vs. K. GraphSeg appears as a horizontal reference because it does not use K directly.

and temporary disk writes (`temp_proc.png`) inside the experiment loop.

The implemented UE variant can exceed 1.0 in coarse settings because each superpixel contributes proportionally to the number of touched GT regions. This behavior is consistent with the implemented formula and still supports reliable relative comparisons across methods/settings.

6. Conclusion and Future Work

Our evaluation of the custom SLIC implementation against established baselines (OpenCV SLIC, SEEDS, GraphSeg) highlights clear trade-offs between segmentation quality and computational efficiency. The results demonstrate a consistent quality trend as K increases: higher boundary recovery, lower leakage, and higher achievable accuracy, albeit with increased computational cost.

Our custom SLIC demonstrates strong segmentation quality, particularly in Boundary Recall and competitive ASA at medium/high K, confirming the effectiveness of our localized color-spatial clustering and connectivity post-processing strategy. Conversely, optimized OpenCV backends maintain a major runtime advantage, which is expected given their compiled nature and specialized routines.

From a practical perspective, this study provides a clear trade-off map. If quality is prioritized, custom SLIC with tuned m and higher K provides strong contour-faithful superpixels. If speed is prioritized, OpenCV methods are preferable for real-time or large-scale deployment. Furthermore, if a fixed non-K baseline is needed, GraphSeg gives a fast and informative reference.

To improve both segmentation quality and runtime efficiency in future iterations, the next development steps include removing temporary disk I/O by passing arrays directly between functions, vectorizing remaining hot loops, and testing Numba/Cython acceleration. Additionally, we could also add a full-split BSDS500 aggregate tables with confidence intervals, include compactness/regularity metrics in addition to BR/UE/ASA, and evaluate adaptive compactness m and edge-aware distance weighting for future improvements.

Overall, this project establishes a solid and reproducible foundation for superpixel research, offering a technically grounded path for future improvements.

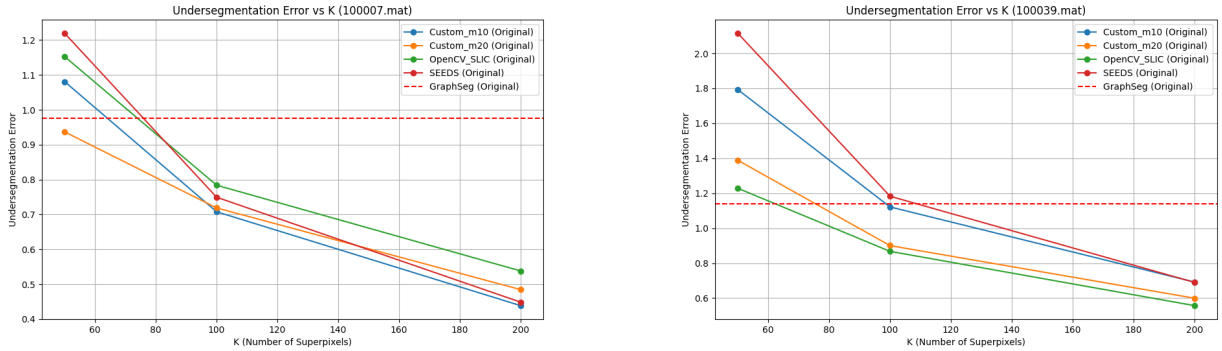

Figure 3. Undersegmentation Error vs. K (lower is better).

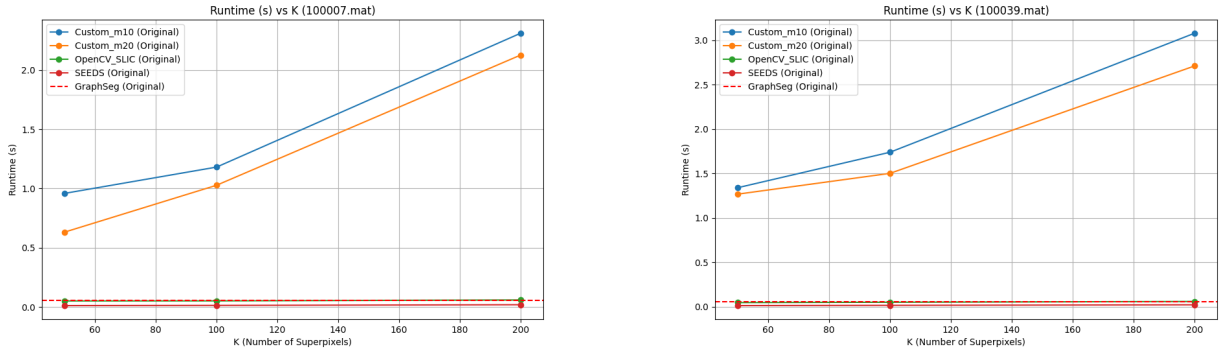

Figure 4. Runtime vs. K. OpenCV backends are substantially faster than the pure-Python custom implementation.

References

[1] Radhakrishna Achanta et al. Slic superpixels compared to state-of-the-art superpixel methods. *IEEE TPAMI*, 2012. 1, 2, 3

[2] Pablo Arbeláez et al. Contour detection and hierarchical image segmentation. *IEEE TPAMI*, 2011. 1

[3] Charles Elkan. Using the triangle inequality to accelerate k-means. In *Proceedings of the International Conference on Machine Learning*, 2003. 4

[4] Pedro F. Felzenszwalb and Daniel P. Huttenlocher. Efficient graph-based image segmentation. *International Journal of Computer Vision*, 59(2):167–181, 2004. 1, 2

[5] Tapas Kanungo, David M. Mount, Nathan S. Netanyahu, Christine D. Piatko, Ruth Silverman, and Angela Y. Wu. A local search approximation algorithm for k-means clustering. In *Proceedings of the Symposium on Computational Geometry*, 2002. 4

[6] Amit Kumar, Yogish Sabharwal, and Sandeep Sen. A simple linear-time (1+e)-approximation algorithm for k-means clustering in any dimensions. In *Proceedings of the IEEE Symposium on Foundations of Computer Science*, 2004. 4

[7] Stuart P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. 4

[8] Peer Neubert and Peter Protzel. Superpixel benchmark and comparison. In *Proceedings of Forum Bildverarbeitung*, 2012. 3

[9] Michael Van den Bergh et al. Seeds: Superpixels extracted via energy-driven sampling. In *ECCV*, 2012. 2